

AI Agent 模拟面试题库

基于 Agent 知识体系整理的 55 道模拟面试题，覆盖 Agent 核心、RAG、工程化、系统设计和 LLM 基础五大模块。难度标注： 中级 / 高级

一、Agent 核心概念 (12 题)

1 1. 请用一句话概括 AI Agent 是什么？它和普通 Chatbot 有什么区别？

1.1 参考答案要点

- **Agent 定义：**Agent 是一个能感知环境、做决策、执行动作的软件系统，LLM 负责推理决策，工具负责执行，记忆负责保存上下文和历史经验。
 - **核心公式：** $\text{Agent} = \text{LLM} + \text{Planning} + \text{Memory} + \text{Tools}$ 。
 - **与 Chatbot 的本质区别：**
 - Chatbot 是”你问它答”，不会主动执行操作；
 - Agent 会在动态环境里持续观察、判断、执行，直到任务结束；
 - Agent 能调用外部工具（API、文件系统、代码执行），不只是生成文字。
 - **设计理念差异：**Chatbot 解决”会说”，Agent 解决”会做”。
 - **架构差异：**Agent 有 Agent Loop（推理 → 行动 → 观察 → 再推理的循环），Chatbot 是单轮请求-响应。
-

2 2. Agent Loop 的完整流程是什么？工程难点在哪里？

2.1 参考答案要点

- **完整流程：**
 1. 初始化：加载 System Prompt、可用工具列表、用户请求；
 2. 循环迭代：读取上下文 → LLM 推理下一步（调用工具 or 直接回复）→ 执行工具 → 将结果追加到上下文；
 3. 退出条件：LLM 判断任务完成，不再调用工具时退出；
 4. 安全兜底：设置最大迭代轮次上限（一般 10~{ }20 轮）或 Token 消耗阈值。
 - **工程难点不在 while 循环本身，而在上下文管理：**
 - 任务越跑越久，上下文越来越长，关键信息被稀释，模型容易跑偏；
 - 这是 Context Engineering 要解决的核心问题；
 - 还需要处理工具调用失败、多轮推理跑偏、Token 成本失控等问题。
-

3 3. ReAct、Plan-and-Execute、Reflection、Multi-Agent 这些范式分别适合什么场景？

3.1 参考答案要点

- **ReAct (Reasoning + Acting)：**
 - 推理和行动交替进行，走一步看一步；
 - 适合路径不确定、需要根据证据调整方向的任务（如故障排查、代码库探索）。

- **Plan-and-Execute:**
 - 先规划整体方案，再逐步执行；
 - 适合步骤较多但可提前规划的复杂任务，”更像 Workflow 和 Agent 的混合体”；
 - 相比 ReAct 更可控，Token 消耗更可预测。
 - **Reflection:**
 - 任务完成后进行自我复盘，提取 Meta-Knowledge；
 - 适合需要持续改进、从失败中学习的场景；
 - 可做三重验证：真实性验证、交付物验证、数据保真性验证。
 - **Multi-Agent:**
 - 多个 Agent 分工协作（如一个负责规划、一个负责执行、一个负责检查）；
 - 适合复杂任务可自然拆解的场景；
 - 存在通信开销、上下文污染、任务一致性问题，生产里不能盲目上。
-

4 4. Tools 注册时，工具 description 为什么非常关键？

4.1 参考答案要点

- **模型只认结构化描述：**LLM通过 Function Calling Schema（JSON Schema 格式）理解工具有什么用、什么时候该用、参数怎么填。
 - **description 决定了两个核心行为：**
 1. **是否调用：**模型根据 description 判断当前场景是否适合调用该工具；
 2. **参数怎么填：**每个参数的 description 告诉模型该字段的含义和合法值范围。
 - **最佳实践：**
 - description 要写清楚工具的适用场景和不适用场景（”什么情况下别调这个”）；
 - 参数描述要给出格式示例（如”时间范围，格式 HH:MM-HH:MM，比如 09:00-09:30”）；
 - 默认值要有明确说明。
 - **错误案例：**description写得模糊，模型会乱调用不相关的工具，造成 Token 浪费和结果偏差。
-

5 5. Agent 的短期记忆和长期记忆有什么区别？分别怎么落地？

5.1 参考答案要点

- **短期记忆（Session 级）：**
 - 当前单次会话中的暂存信息：用户提问、模型回复、工具调用中间结果；
 - 依托 LLM 上下文窗口，会随 Session 结束而消失；
 - 落地手段：滑动窗口裁剪、上下文摘要压缩、上下文卸载（大结果存外部，Prompt 只放引用）。
 - **长期记忆（跨 Session 级）：**
 - 持久化知识库：用户偏好、历史决策、过往经验；
 - 通过”写入-检索”机制让新 Session 还能拿到沉淀数据；
 - 落地手段：向量库 + 语义检索 + Reranker 精排，必要时结合图数据库（Neo4j）做多跳推理。
 - **核心差异：**短期是”刚才说过什么”，长期是”过去积累了什么”。两者物理和逻辑上都应分开管理。
-

6 6. 记忆系统需要解决哪些核心问题？如何避免记忆污染？

6.1 参考答案要点

- **核心问题清单：**

1. 记忆压缩：上下文窗口有限，历史对话需要压缩，但压缩会丢失细节（如精确错误码、关键数值）；
2. 记忆过期：旧偏好和事实可能已不再适用，需要失效机制；
3. 记忆冲突：新事实和旧记录矛盾，需要冲突检测与合并；
4. 记忆检索准确性：向量检索不是精确匹配，语义相近不等于相关。

- **工程策略：**

- 权重衰减公式： $\text{score} = \text{relevance} \times \text{importance} \times \text{decay}(\text{time})$ ，低分记忆自动淘汰；
 - 边失效机制（ZEP）：新事实和旧边时间重叠时，标记旧边失效并打时间戳；
 - LLM 抽取时加校验：JSON Schema 强约束输出、LLM-as-Judge 二次校验、低置信度不写入；
 - 幂等 Key 设计：基于源消息 ID + 抽取批次 ID，避免重试产生重复记忆。
-

7 7. 向量记忆和 Markdown 记忆分别适合什么场景？

7.1 参考答案要点

- **向量记忆：**

- 适合语义模糊匹配（“用户喜欢简洁的风格”）；
- 基于 Embedding + 向量检索，召回率高但不精确；
- 适用场景：偏好检索、经验召回、相似情景匹配。

- **Markdown 记忆：**

- 适合结构化规则和确定性知识（“该用户的 Java Code Review 优先关注 OOM 风险”）；
- 可人工编辑、可 Git 版本管理、可 Code Review；
- 适用场景：团队共享知识、Skill 文档、规则库。

- **选型建议：**

- 不确定的、需要语义匹配的 → 向量记忆；
 - 确定的、需要人审核的 → Markdown 记忆；
 - 二者不是互斥的，可以混合使用。
-

8 8. 为什么 Agent 场景下只优化 Prompt 不够？Context Engineering 解决什么问题？

8.1 参考答案要点

- **Prompt Engineering 的边界：**

- 解决“怎么把指令说清楚”的问题，聚焦于单次模型调用的输入质量；
- 在简单任务里效果显著，但在长链路 Agent 场景里作用递减。

- **Context Engineering 解决的问题：**

- 在合适时机给模型提供正确且必要的信息；
- 核心问题：静态规则、动态信息、工具结果、记忆应该如何进入上下文；
- 具体手段：Token 降级（JIT 卸载）、上下文隔离（多 Agent 不广播全量历史）、上下文利用率监控（40% 告警线）。

- **为什么 Prompt 不够：**

- Agent 是多轮循环，不是单次调用，信息在循环中持续累积和稀释；
 - Prompt 调得再好，上下文混乱模型一样跑偏；
 - 长任务中上下文溢出时，需要 Compaction（压缩）、结构化笔记、Sub-agent 分流等手段，这些都不是 Prompt Engineering 能解决的。
-

9 9. MCP 协议解决什么问题？它和 Function Calling 有什么区别？

9.1 参考答案要点

- **MCP 解决的问题：**
 - 工具接入碎片化：每个 LLM 平台各自适配工具，维护成本高；
 - 提供统一的 Client-Server 协议，让工具以标准化方式被发现和调用；
 - 被类比为“AI 领域的 USB-C”。
 - **三者关系：** MCP Client（使用者，如 Agent）→ MCP Server（提供者，暴露工具）→ Host（运行环境）。
 - **MCP 的核心能力：** Tools（执行操作）、Resources（提供数据）、Prompts（模板）。
 - **与 Function Calling 的区别：**
 - **Function Calling** 解决数据格式问题（OpenAI Schema 描述工具名称/参数/类型）；
 - **MCP** 解决通信接入问题（工具如何被发现、连接、调用）；
 - 二者互补：Function Calling 定义“工具长什么样”，MCP 定义“怎么连上工具”。
 - **生产级 MCP Server 安全治理：** 权限、审计、限流、脱敏、人工确认都要补上。
-

10 10. Agent Skills 是什么？Skill 路由怎么做？

10.1 参考答案要点

- **Skills 定义：**
 - 很多任务不是一句 Prompt 能做好，而是需要固定流程、上下文、模板、脚本和校验规则；
 - Skill 把这些方法封装下来，让 Agent 遇到类似任务时按既定流程执行。
 - **Skills 和 Prompt / MCP / Function Calling 的边界：**
 - Prompt 是“怎么说”，Skill 是“怎么做一套”；
 - Function Calling 是单次工具调用，Skill 是工具 + 流程 + 上下文 + 校验的组合；
 - MCP 是工具接入协议，Skill 是更高层的任务能力封装。
 - **Skills 为什么要延迟加载：**
 - 用 description 做路由匹配，命中后再加载 SKILL.md 正文；
 - 不需要每轮都把规则重新塞进上下文，避免撑爆上下文窗口；
 - 这也叫“渐进式披露”（Progressive Disclosure）。
 - **Skill 路由：** 本质是语义匹配问题，和 RAG 相似但目标不同——RAG 检索知识，Skill 检索执行流程。
 - **写 SKILL.md 的常见坑：**
 - 写得像超级 System Prompt，把所有规则塞进去，上下文被撑爆；
 - 没有写清除边界（什么时候该用、什么时候不该用）；
 - 没有写失败处理和回滚规则。
-

11 11. Multi-Agent 协作的主要问题是什么？为什么生产里不能盲目上多 Agent？

11.1 参考答案要点

- **主要问题：**
 1. 上下文污染：子 Agent 拿到不相关的历史信息，越跑越偏；
 2. 通信开销：Agent 之间的消息传递消耗大量 Token；
 3. 任务一致性：多个 Agent 对同一任务的理解可能不一致，产出矛盾；
 4. 局部最优陷阱：每个 Agent 都在自己的局部范围内做最优决策，组合起来全局可能很差；
 5. 调试困难：多 Agent 的决策链路复杂，排查问题困难。

- **生产建议：**
 - 先用单 Agent + 良好 Harness 把核心链路跑稳；
 - 只在任务天然可拆解（如”规划 → 执行 → 检查”）时才引入多 Agent；
 - 多 Agent 架构必须做上下文隔离，主 Agent 只给子 Agent 传精简指令；
 - DAG 编排（如 Coze、Dify）比完全自治的多 Agent 更可控。
-

12 12. 上下文利用率为什么有一个 40% 的阈值现象？怎么利用这个现象做工程优化？

12.1 参考答案要点

- **40% 阈值现象 (Smart Zone / Dumb Zone)：**
 - 来源：Dex Horthy 观察到 168K token 上下文窗口，用到约 40% 时 Agent 输出质量开始明显下降；
 - Smart Zone (0~40%)：推理聚焦、工具调用准确、代码质量高；
 - Dumb Zone (>40%)：幻觉增多、兜圈子、格式混乱、代码变差。
 - **原因分析：**
 - Lost in the Middle：模型对上下文首尾信息利用效率高，中间段利用率明显更低；
 - 上下文越长，位置偏差越明显，噪声越多。
 - **工程优化：**
 - 把 40% 当成告警线，超过后触发压缩、分段执行或任务交接；
 - 用”渐进式披露”和”分层管理”，只让模型看到当前任务相关的上下文；
 - Context Reset：清空上下文窗口，但通过结构化交接文档保留关键状态；
 - 在生产环境中监控上下文利用率，被动等待 Agent 跑偏再处理已经晚了。
-

二、RAG 检索增强 (8 题)

13 13. 什么是 RAG？它和传统搜索引擎、微调有什么区别？

13.1 参考答案要点

- **RAG 定义：** Retrieval-Augmented Generation，在 LLM 生成回答前，先从外部知识库检索相关内容，注入上下文后再生成。
 - **解决什么问题：**
 - LLM 知识陈旧（训练数据有截止日期）；
 - 无法访问企业内部私有数据；
 - 减少幻觉，提供可溯源的引用。
 - **与传统搜索引擎的区别：**
 - 搜索引擎返回链接列表，用户自己判断和阅读；
 - RAG 检索后由 LLM 自动阅读理解、融合、生成最终答案；
 - RAG 更省去用户自己查找、比对、总结的步骤。
 - **与微调的决策：**
 - **RAG：** 知识频繁更新、需要可溯源的引用、私有数据不能用于训练；
 - **微调：** 需要改变模型的行为风格或特定领域推理模式、知识相对稳定；
 - **两者结合：** 用微调让模型学会”怎么用检索结果”，用 RAG 提供实时知识。
-

14 14. RAG 系统中 Chunk Size 怎么选？太大和太小分别有什么影响？

14.1 参考答案要点

- **Chunk Size 的 Tradeoff:**
 - **太小** (如 128 tokens): 语义完整性差, 碎片化严重, 检索召回率高但准确率低;
 - **太大** (如 2048 tokens): 包含过多无关信息, 检索精度下降, 且会挤占上下文窗口;
 - **合适范围:** 一般 256~1024 tokens, 需要根据文档类型和数据分布做 A/B 测试。
 - **工程经验:**
 - 技术文档适合中等 chunk (~512 tokens), 保留完整段落语义;
 - FAQ 类短文本适合小 chunk (~256 tokens);
 - 长文分析类适合大 chunk (~1024 tokens) 或 Sentence Window Retrieval。
 - **进阶策略:**
 - 父子文档 (Parent-Child): 小 chunk 检索 + 大 chunk 返回上下文;
 - 滑动窗口分块: 相邻 chunk 之间有重叠, 减少语义断裂;
 - 上下文扩展: 检索到的 chunk 自动带上前后文。
-

15 15. 余弦相似度、内积、欧氏距离分别适合什么 Embedding 模型？

15.1 参考答案要点

- **余弦相似度:**
 - 比较向量方向, 对模长不敏感;
 - 适合大多数经过 L2 归一化的 Embedding 模型 (如 OpenAI text-embedding-ada-002);
 - 最通用的选择。
 - **内积 (Dot Product):**
 - 同时考虑方向和模长;
 - 适合未归一化的 Embedding 或需要区分”强相关”和”弱相关”的场景;
 - 模长通常和信息量正相关。
 - **欧氏距离:**
 - 对模长敏感;
 - 在文本相似度场景下使用较少, 但适合需要空间可解释性的场景;
 - 配合 L2 归一化后等价于余弦相似度。
 - **关键决策点:** 先确认 Embedding 模型的输出是否归一化, 再选择相似度度量。
-

16 16. 向量索引 HNSW、IVF、DiskANN 分别适合什么场景？

16.1 参考答案要点

- **HNSW (Hierarchical Navigable Small World):**
- 图索引, 召回率高 (Recall@10 > 0.95), 内存占用大;
- 适合低并发 + 高精度场景 (如 QPS < 50, P99 数十毫秒);
- 元数据过滤率高时 pre-filter 可能破坏图连通性, 需要关注。
- **IVF (Inverted File Index):**
- 聚类索引, 先粗聚类再精确搜索;
- 内存占用小, 适合百万级以上数据量;
- 召回率略低于 HNSW, 但参数调优 (nprobe) 可以折中。
- **DiskANN:**

- 基于 SSD 的图索引，内存占用极小；
 - 适合十亿级数据量 + 低内存预算的场景；
 - 延迟比 HNSW 高，但性价比更好。
 - **选型维度**：索引类型、元数据过滤方式、多租户隔离、持久化一致性、成本模型。
-

17 17. RAG 评测为什么必须分检索和生成两段？怎么打分？

17.1 参考答案要点

- **为什么要分段评测**：
 - 检索和生成是独立环节，问题可能出在任一环节；
 - 检索不好 → 给 LLM 的上下文本身就是错的，生成再好也没用；
 - 生成不好 → 检索对了但 LLM 没用对信息；
 - 混在一起评测无法定位问题根源。
 - **检索段评测指标**：
 - Recall@K：Top-K 结果中召回相关文档的比例；
 - MRR (Mean Reciprocal Rank)：第一个相关文档的排名倒数均值；
 - NDCG：考虑排序质量的归一化折损累计增益。
 - **生成段评测指标**：
 - 忠实度 (Faithfulness)：生成内容是否基于检索到的文档，有无编造；
 - 相关性 (Relevance)：回答是否切中问题；
 - 准确性 (Correctness)：事实是否正确。
 - **评测方法**：
 - 构建 Golden Set (标准问答对 + 标注相关文档)；
 - 使用 LLM-as-Judge 自动评测，配合人工抽检校准。
-

18 18. 长上下文窗口会不会取代 RAG？什么时候用 RAG，什么时候直接用长上下文？

18.1 参考答案要点

- **长上下文不取代 RAG 的原因**：
 1. **成本**：上下文越长，推理成本线性增长；RAG只注入精选片段，Token 更省；
 2. **准确性**：Lost in the Middle问题在长上下文中更严重，中间信息利用率低；
 3. **信息密度**：RAG做了检索和排序，给模型的是高密度相关信息，而非海量原始文本；
 4. **知识更新**：长上下文不能实时更新知识源，RAG可以随时切换向量库。
 - **选型决策**：
 - 用 **RAG**：知识量远超上下文窗口、需要精确溯源、知识频繁更新；
 - 用**长上下文**：单文档深度分析（如合同审查）、需要理解全文结构、检索精度难以保证时；
 - **两者结合**：RAG先粗筛，长上下文再精读，是常见的最佳实践。
-

19. 多模态 RAG 相比纯文本 RAG 多了哪些挑战？

19.1 参考答案要点

- 核心挑战：

1. Embedding 模型不统一：文本用文本 Embedding，图片用 CLIP 等多模态 Embedding，分属不同向量空间；
2. Chunk 策略复杂：一个 PPT 页里图表 + 文字 + 表格混排，怎么拆？图文关联怎么保留？
3. 检索结果融合：用户纯文本 query，怎么召回图片？怎么把图文结果一起排序？
4. 多模态生成：LLM 能否理解图表并给出文字回答？需要多模态 LLM 支持。

- 工程应对：

- 多向量库并存：文本库 + 图片库，用路由决定查哪个；
 - 图文关联：图片 Chunk 里带上周围的文字描述或 Caption；
 - 检索策略：分别检索后做结果融合（Fusion）或交叉排序；
 - 多模态 LLM：用 GPT-4V / Claude 等多模态模型直接理解图文混合上下文。
-

20. RAG 和 Agent 的记忆系统在技术上有哪些异同？

20.1 参考答案要点

- 相同之处：

- 都用向量库和语义检索做相似性匹配；
- 都有“写入-检索”两条链路；
- 都需要处理检索精度和排序问题（Reranker）。

- 本质区别：

- RAG 挂载的是共享知识源（公司章程、产品文档），非个性化，对不同用户返回同样内容；
 - 长期记忆管理的是 Agent 与特定用户的交互中沉淀的个性化经验（偏好、习惯、历史决策），高度个性化。
 - 配合使用：
 - RAG 提供“世界知识”，长期记忆提供“用户画像”；
 - 检索阶段可以分别召回再融合排序；
 - 长期记忆里的实体可作为 RAG 检索的 query 扩展；
 - 用户偏好可作为 RAG 结果的个性化重排信号。
-

三、Agent 工程化（12 题）

21. Harness Engineering 是什么？为什么说 Agent = Model + Harness？

21.1 参考答案要点

- Harness 定义：

- 指模型之外的整套系统：系统提示词、工具调用、文件系统、沙箱环境、编排逻辑、钩子中间件、反馈回路、约束机制；
- 模型只提供推理和生成能力，Harness 把状态、工具、反馈、执行环境和安全边界串起来。

- Agent = Model + Harness 的含义：

- 模型像 CPU，Harness 像操作系统——CPU 再强，OS 天天崩，体验也不会好；
- 同一个模型，只改变 Harness（如文件编辑接口的调用方式），编码基准分数可从 6.7% 跳到 68.3%。

- 为什么瓶颈经常不在模型：

- 工具接口设计得难用 → 模型调不对；
- 反馈回路缺失 → 模型不知道自己错了；

- 上下文太杂乱 → 模型输出质量下降；
- 调 Prompt 解决不了这些问题。

22 22. Harness 的六层架构分别解决什么问题？

22.1 参考答案要点

层级	名称	解决的问题	关键设计
L1	信息边界层	Agent 该知道什么、不该知道什么	定义角色与目标，裁剪无关信息
L2	工具系统层	Agent 怎么和外部世界交互	选择工具、控制调用时机、提炼工具结果
L3	执行编排层	多步骤任务怎么串起来	理解目标 → 判断信息 → 分析 → 生成 → 检查
L4	记忆与状态层	长任务中间结果怎么管理	独立管理任务状态、中间产物和长期记忆
L5	评估与观测层	Agent 怎么知道自己做对了没有	独立于生成过程的验证机制
L6	约束、校验与恢复层	出错了怎么办	预设规则拦截错误，重试、回滚或降级

- 类比：L1= 岗位说明，L2= 办公工具，L3= 标准操作流程，L4= 项目管理系统，L5= 质检，L6= 红线规则。
- 实施建议：不要一上来全搭齐，先做 L1（边界）和 L6（兜底），这两层投入不高但最容易见效。

23 23. 为什么同一个模型换一套工具接口，效果能差 10 倍？

23.1 参考答案要点

- 实验证据：can.ac实验，同一个模型，只换了文件编辑接口的调用方式，编码基准分数从 6.7% 跳到 68.3%。
- 原因分析：
 1. 工具返回信息的格式和精度直接影响模型的下一步判断；
 2. 错误信息有没有给出修复方向，决定了模型能否自我纠正；
 3. 工具之间的衔接（如编辑后自动运行 lint，lint 结果自动反馈）形成有效闭环；
 4. 工具接口如果把无关信息也塞给模型，等于在被污染的信息上做推理。
- model-harness 耦合现象：
 - 模型和 Harness 一起调优会过拟合——模型习惯了某套工具逻辑；
 - 换一个 Harness 表现可能变差；
- 结论：the best harness for your task is not necessarily the one a model was post-trained with。

24 24. Loop Engineering 是什么？它和 Agent Loop、Workflow Graph 有什么区别？

24.1 参考答案要点

- Loop Engineering 定义：
 - 围绕 Agent 设计可持续运行的反馈循环，让其在明确目标、工具、上下文、验证信号和停止条件下反复行动；
 - 关注的不再是”一句 Prompt 怎么写”，而是”谁来触发、带哪些材料、跑完拿什么证据判断”。
- 与 Agent Loop（内层循环）的区别：
 - Agent Loop：Agent 自己每一轮”推理 → 行动 → 观察”的循环；
 - Engineering Loop（外层循环）：调度系统或人写的流程，唤醒 Agent、分配任务、验证结果、记录状态。
- 与 Workflow Graph 的关系：
 - Loop 就是 Graph 里的回边（”生成初稿 → 审核 → 不通过继续改”）；
 - Loop Engineering 把这种循环拓展到跨 Claude Code、CI、GitHub、Issue 系统的日常工作中。
- 核心要素：触发、目标、上下文、行动、观察、状态、停止条件。

25 25. Workflow、Graph、Loop 三者是什么关系？什么场景用哪种编排方式？

25.1 参考答案要点

- **关系：**
 - Workflow 是最外层概念，描述“任务怎么做”的流程；
 - Graph 是 Workflow 的形式化表示（节点 + 边），可以用 DAG（有向无环图）约束执行流程；
 - Loop 是 Graph 里的回边（条件边），让流程可以循环执行。
 - **选型：**
 - **纯 Workflow (DAG)：** 流程清晰、步骤有限、需要可视化管理和审计（如审批流、数据管道）；
 - **Agent Loop (ReAct)：** 步骤不确定、需要动态判断的任务（如故障排查）；
 - **Plan-and-Execute：** 超长流程里夹杂动态子任务，Workflow 和 Agent 的混合体；
 - **多 Agent Graph：** 任务天然可拆解为并行子任务，由编排层管理 Agent 间消息传递。
 - **Graph Loop 的安全设计：** continue 条件、exit 条件、最大轮次、超时、Token 预算、失败降级。
-

26 26. 如何防止 Agent Loop 陷入死循环？

26.1 参考答案要点

- **多层防护策略：**
 1. 最大迭代轮次上限（硬限制，一般 10~20 轮）；
 2. Token 消耗上限（防止烧钱）；
 3. 超时限制（防止单轮工具调用挂死）；
 4. 重复检测：如果连续 N 轮调用相同工具且参数相同 → 终止；
 5. 进展检测：如果工具返回结果连续无有效产出 → 终止。
 - **工程实现：**
 - 每轮 loop 记录工具调用和结果摘要；
 - 设置全局状态变量跟踪任务进展；
 - 超出限制时，保存部分结果并抛回给用户或转人工。
 - **与 Graph Loop 的区别：** Graph Loop 条件更可控（条件边 + 状态机），Agent Loop 更依赖 LLM 自己的判断。
-

27 27. 大模型网关需要承担哪些核心能力？

27.1 参考答案要点

- **核心能力清单：**
 1. **多模型路由：** 根据任务类型、成本、延迟要求路由到不同的模型（如简单任务用小模型，复杂任务用大模型）；
 2. **Fallback 与容错：** 主模型不可用时自动切换到备用模型；
 3. **限流与流控：** 同时看 RPM、TPM、并发数和租户预算四维限流；
 4. **成本控制：** 按 Token 用量、模型类型、租户做成本归因和预算告警；
 5. **PII 脱敏：** 在请求进入模型前做敏感信息检测和脱敏处理；
 6. **审计与可观测：** 记录每次调用的模型、Token 用量、延迟、错误码；
 7. **协议统一：** 屏蔽不同模型 API 的差异，对上游暴露统一接口。
 - **为什么不能只按 QPS 限流：**
 - Token 消耗不均：一次调用可能 10 token 也可能 10K token；
 - TPM 才是真正的吞吐瓶颈；
 - 多租户预算隔离需要租户级别的限流维度。
-

28. 模型 Fallback 策略怎么设计？什么时候不能自动降级？

28.1 参考答案要点

- **Fallback 层级设计：**
 1. 同厂商同模型重试（瞬态错误）；
 2. 同厂商降级模型（主模型不可用，用同厂小模型）；
 3. 跨厂商切换（切换到另一个云厂商的同等能力模型）；
 4. 静态/缓存回答（所有模型都不可用时的兜底）。
 - **什么时候不能自动降级：**
 - 安全/合规场景（模型回答涉及法律、医疗等高风险决策）；
 - 结构化输出场景（小模型不保证 JSON Schema 合规）；
 - 工具调用安全要求高（高风险操作必须有二次确认，不能换模型静默执行）；
 - 质量敏感场景（降级模型可能导致错误决策的代价高于不可用的损失）。
 - **降级策略必须配合：**语义一致性校验、质量阈值、人工审核队列。
-

29. Token 成本怎么归因到租户、用户、功能和 Prompt 版本？

29.1 参考答案要点

- **归因维度设计：**
 - **租户维度：**多租户平台必须按租户拆分 Token 用量；
 - **用户维度：**租户内部的用户级别的消费统计；
 - **功能维度：**不同功能模块（对话、RAG检索、代码生成、Agent 任务）分别统计；
 - **Prompt 版本维度：**不同 Prompt 版本的 Token 效率对比，用于优化。
 - **工程实现：**
 - 每次 API 调用日志记录：`{tenant_id, user_id, feature, prompt_version, model, input_tokens, output_tokens, cost}`；
 - 实时聚合 + 离线批处理两条线：实时用于限流和预算告警，离线用于分析和优化；
 - 带上异步任务的 Token 消耗（如记忆抽取、检索、反思等后台任务）。
 - **常见陷阱：**只算大模型 API 的 Token 成本，忘记算 Embedding、Reranker、向量库查询的算力成本。
-

30. Agent 的 AGENTS.md 应该怎么写？为什么不要写成超级 System Prompt？

30.1 参考答案要点

- **AGENTS.md 的定位：**
 - Agent 每次启动自动加载，是项目级的知识入口；
 - 应该像目录，而不是把所有规则塞进去。
- **OpenAI 的最佳实践：**
 - AGENTS.md 大约 100 行，当目录用；
 - 详细规则放到子文档里按需加载（渐进式披露）；
 - 每一行对应一个历史失败案例，犯错后更新，形成反馈循环。
- **为什么不能写成超级 System Prompt：**
 - 上下文窗口有限，全塞进去会进入 Dumb Zone（>40% 利用率质量下降）；
 - Agent 面对臃肿的规则更容易“迷路”；
 - 规则混在一起，Agent 不会区分什么场景用什么规则。
- **写法建议：**

- 项目基本信息（技术栈、目录结构、关键约定）；
 - 规则索引（指向子文档）；
 - 禁止事项（红线规则）；
 - 常见错误和修复方法。
-

31 31. workflow中断后怎么恢复？State 更新策略怎么选？

31.1 参考答案要点

- **中断恢复机制：**
 1. 检查点（Checkpoint）：每个关键节点完成后保存 State 快照；
 2. 幂等重放（Replay）：从中断前的最近检查点回放执行；
 3. 人工接管：中断后保存当前状态和上下文，支持人工介入后继续。
 - **State 更新策略选择：**
 - **Replace（替换）**：适合不变的事实型状态，如用户信息、任务目标；
 - **Append（追加）**：适合过程记录型状态，如执行日志、对话历史；
 - **Reducer（归约）**：适合需要合并累加的状态，如 Token 消耗累计、进度百分比。
 - **实现要点：**
 - State 要持久化到外部存储（数据库、文件），不能只依赖内存；
 - 每步更新后原子性写入，避免中断时状态不一致；
 - State 结构要兼顾人可读（调试）和机器可解析（Schema 约束）。
-

32 32. AI 应用的可观测性要看哪些指标？

32.1 参考答案要点

- **核心指标分类：**
 1. **延迟指标**：TTFT（首 Token 时间）、端到端延迟、工具调用延迟、检索延迟；
 2. **质量指标**：任务成功率、幻觉率、用户反馈评分、LLM-as-Judge评分；
 3. **成本指标**：Token消耗、API 调用次数、异步任务开销；
 4. **可靠性指标**：API错误率、超时率、Fallback 触发次数、循环异常终止率；
 5. **安全指标**：PII检出次数、Prompt 注入尝试次数、高风险操作审计。
 - **追踪（Tracing）设计：**
 - 每次请求生成 traceId，贯穿 Agent Loop 的每一轮；
 - 记录每轮的输入上下文、LLM 输出、工具调用参数和结果；
 - 用于事后排查和 Trace 回放评测。
 - **告警设计：**
 - 上下文利用率超过 40% → 告警；
 - 任务成功率连续下降 → 告警（可能是模型/Prompt/检索出现变化）；
 - Token 消耗异常飙升 → 告警。
-

四、系统设计（12 题）

33 33. 请设计一个生产级 AI 应用的整体架构，从入口到模型返回的完整链路是什么？

33.1 参考答案要点

- 六层架构（自外向内）：

层级	职责
入口层	API Gateway、认证鉴权、限流、请求路由
编排层	理解用户意图 → 判断是否需要 RAG/Agent/Tool → 编排执行计划
Prompt/Context 层	组装 System Prompt、注入 RAG 结果、注入记忆、Token 预算控制
RAG/Memory/Tool 层	检索知识、召回记忆、执行工具调用、结果提炼
模型网关层	多模型路由、Fallback、重试、限流、PII 脱敏、审计
评测观测层	日志/Trace 采集、质量评测、成本归因、告警

- 请求链路：
 1. 用户请求进入 API Gateway → 鉴权 + 限流；
 2. 编排层分析意图，决定走简单问答/RAG/Agent Loop；
 3. Context 层注入相关记忆 + RAG 结果；
 4. 如需工具调用，Agent Loop 执行推理 → 调用 → 观察循环；
 5. 模型网关选模型、发请求、处理重试和降级；
 6. 结果经过安全校验后返回用户，同时写入日志和 Trace。

34 34. 同步、流式（SSE）、异步三种模式怎么选？

34.1 参考答案要点

模式	适合场景	优缺点
同步（Request-Response）	短回复、低延迟要求	实现简单，用户等待时间长
流式（SSE）	对话式 AI、逐字生成	显著改善体感（TTFT 低），不能减少总耗时，需处理断连重连
异步（任务队列）	长任务（Agent、报告生成）	用户不阻塞，需轮询或 Webhook 通知结果，复杂度高

- 选型建议：
 - 简单问答 → SSE 流式输出；
 - Agent 长任务 → 异步模式，任务提交后返回 taskId，结果通过 WebSocket 推送或轮询获取；
 - WebSocket vs SSE：WebSocket 双向通信，适合语音 Agent 等需要双向推送的场景；SSE 更轻量，适合纯单向流式文本。

35 35. 为什么大模型调用限流要同时看 RPM、TPM、并发数和租户预算？

35.1 参考答案要点

- 为什么不能只按 RPM 限流：
 - RPM（Request Per Minute）只控制了请求频率；
 - 一次请求可能消耗 10 token，也可能消耗 10 万 token；
 - 短请求高频 vs 长请求低频，RPM 相同的 TPM 可能差 100 倍。
- 四维限流体系：
 - RPM：控制 API 调用频率（防止连接数爆炸）；
 - TPM：控制真正的吞吐瓶颈（Token Per Minute）；
 - 并发数：控制同时正在处理的请求数（保护后端服务）；
 - 租户预算：控制每个租户的成本上限，防止单租户超预算。

- **工程实现：**
- 分层限流：全局 → 租户 → 用户 → 功能；
- Token 计算器在每次请求前后原子性更新；
- 超限时返回 429 + Retry-After，而非直接丢弃。

36 36. AI 应用如何做超时、重试、限流、熔断和降级？

36.1 参考答案要点

- **超时策略：**
- 分级超时：LLM API 调用 30s，工具调用按工具类型设不同超时；
- 流式请求用首 Token 超时（如 5s 没收到首 Token 就 Fallback）。
- **重试策略：**
- 可重试：网络错误、429 限流、5xx 临时故障；
- 不可重试：4xx 参数错误、402 余额不足、内容安全拦截；
- 退避策略：指数退避 + 随机抖动 (jitter)。
- **熔断：**
- 错误率超过阈值 → 熔断器打开，快速失败；
- 半开状态：熔断后定期试探性调用，成功则恢复正常。
- **降级：**
- 模型降级：大模型 → 小模型 → 静态规则回答；
- 功能降级：去掉 RAG 检索直接用模型知识回答；
- 结果降级：无模型可用时返回缓存或预设兜底信息。

37 37. 从 Prompt Demo 到生产级架构，最大的差距在哪？要补齐哪些能力？

37.1 参考答案要点

- **最大差距：**Demo只验证”能不能跑”，生产需要”跑得稳、跑得对、跑得划算、跑得安全”。
- **要补齐的能力（按优先级）：**

阶段	补齐内容
P0（必须）	错误处理与重试、Prompt 版本管理、基本日志
P1（重要）	模型网关（路由 +Fallback+ 限流）、RAG 检索优化、评测体系 (Golden Set + LLM-as-Judge)
P2（增强）	记忆系统、工具调用安全治理、成本归因、A/B 实验、可观测 (Trace + Metrics)

- **架构演进路径：**
- Prompt Demo → 加上错误处理 → 引入 RAG → 模型网关 → Agent Loop → Harness → 完整评测体系；
- 核心原则：MVP 先跑通，再逐步补齐非功能需求。

38 38. Prompt 注入攻击在系统设计层面怎么防？

38.1 参考答案要点

- **多层防御策略：**
- 1. **输入层：**检测和过滤可疑的 Prompt 注入模式（如”忽略之前的指令””你现在是 DAN”）；
- 2. **权限层：**工具调用最小权限，高风险操作必须二次人工确认；
- 3. **隔离层：**不可信输入和执行环境隔离（沙箱），Agent代码执行在受限容器中；
- 4. **上下文层：**System Prompt使用特殊分隔符，明确区分”系统指令”和”用户输入”；

5. **审计层**：记录所有 Prompt 注入检测日志和高风险操作审计。

- **常见注入模式**：

- 直接覆盖：Ignore all previous instructions and...
 - 间接注入：通过检索到的文档或网页内容注入（RAG 场景）→ 需要对检索结果做安全过滤；
 - 工具结果注入：攻击者控制的 API 返回恶意指令 → 工具结果也需要安全校验。
 - **局限**：Prompt注入没有完美防御方案，需要在安全性和可用性之间做权衡。
-

39. 如何设计一个实时语音 Agent 系统？

39.1 参考答案要点

- **核心组件**：
 - **ASR**：语音转文字（Whisper、Deepgram 等）；
 - **LLM**：文本推理和决策（GPT-4o、Claude 等）；
 - **TTS**：文字转语音（ElevenLabs、Edge TTS 等）；
 - **VAD**：语音活动检测，判断用户是否在说话。
 - **端到端延迟来源**：ASR延迟 + LLM 首 Token 延迟 + TTS 首音频延迟 + 网络传输延迟。
 - **状态管理**：
 - listening（监听中）→ thinking（推理中）→ speaking（播报中）→ interrupted（被打断）；
 - 用户打断时：取消 TTS 播放 → 取消 LLM 生成 → 更新上下文为”用户打断了上次回答”；
 - 使用 WebSocket 维护双向全双工通道。
 - **架构选择**：
 - **云端 API**：延迟在 500ms-2s，成本可控，适合大多数场景；
 - **端云混合**：ASR和 TTS 在本地，LLM 在云端，延迟可低至 200-800ms；
 - **Speech-to-Speech API**：跳过 ASR/TTS 文本中间层，延迟最低但灵活度也最低。
 - **可观测指标**：端到端延迟、打断响应延迟、ASR准确率、对话成功率、用户满意度。
-

40. 出现模型输出事故后，如何通过 Trace 回放定位问题？

40.1 参考答案要点

- **Trace 应该记录的信息**：
 - 每轮 Agent Loop：输入上下文、LLM 请求参数（model、temperature、messages）、LLM 原始输出；
 - 每次工具调用：工具名、入参、返回结果、耗时；
 - 每次检索：query、检索结果（文档 ID、chunk、相似度分数）；
 - 最终输出：经过后处理的返回给用户的回答。
 - **回放排查步骤**：
 - 1. 从用户反馈的 traceId 找到本次请求的完整链路；
 - 2. 检查 LLM 输出：幻觉？格式错误？工具调用参数填错？
 - 3. 检查检索结果：检索到的文档是否相关？是否包含了错误信息？
 - 4. 检查上下文：上下文是否已超过 40% 阈值？是否有之前工具结果污染？
 - 5. 定位根因后，用相同输入回放测试，验证修复方案；
 - 6. 将本次事故加入 Golden Set 作为回归用例。
 - **系统要求**：Trace必须能覆盖全链路，不能只有 LLM 调用的日志。
-

41 41. 如何构建 Golden Set？冷启动阶段没有生产日志怎么办？

41.1 参考答案要点

- **Golden Set 的覆盖维度：**
 - 正常路径 (Happy Path)：最常见的用户场景；
 - 边缘场景 (Edge Cases)：边界输入、模糊表达、长尾问题；
 - 对抗样本 (Adversarial)：试图注入、越权、误导的输入；
 - 高权重失败 (High-Stakes)：答错后果严重的场景。
 - **冷启动策略：**
 1. 人工构造：产品经理 + 领域专家编写 50~100 条核心问答对；
 2. 文档采样：从知识库文档中抽取代表性段落，构造对应的问句；
 3. 历史数据迁移：如果已有旧的客服记录或 FAQ，清洗后使用；
 4. 红队测试：安全团队构造对抗样本，测试边界行为。
 - **持续迭代：**
 - 将生产中发现的新问题持续加入 Golden Set；
 - 定期人工抽检 LLM-as-Judge 评分，校准评测模型偏差；
 - Golden Set 要版本管理，A/B 实验对照。
-

42 42. LLM-as-Judge 有哪些主要偏差？怎么校准？

42.1 参考答案要点

- **主要偏差类型：**
 1. 位置偏差：更倾向排在前面的候选答案；
 2. 冗长偏好：更容易给长答案高分，即使信息密度不高；
 3. 自信偏好：语气更自信的回答评分更高，即使事实错误；
 4. 自我增强偏差：更偏向和自身训练数据风格一致的答案；
 5. 顺序效应：评测顺序影响打分，前面的更容易得高分或低分。
 - **校准方法：**
 - 答案位置随机交换后评两次，取平均；
 - 评测 rubric 要具体，不用笼统的“好/不好”，而是按维度打分（忠实度、相关性、准确性）；
 - 人工抽检 10%~20% 的评测结果，计算人工与 LLM-as-Judge 的一致性；
 - 不一致超过阈值时，调整评测 Prompt 或 rubric；
 - 多 LLM 交叉评测，取投票结果。
 - **CI 中的平衡：**评测集大小和评测频率要控制，避免 CI 时间过长和 API 成本过高。
-

43 43. Agent 评测为什么比普通 RAG 和问答评测复杂得多？

43.1 参考答案要点

- **复杂性来源：**
 1. **多步推理：**Agent 不是单次问答，而是一个序列决策过程，每一步都可能出错；
 2. **工具调用链：**不仅评最终结果，还要评工具选择是否正确、调用参数是否合理、工具结果是否被正确利用；
 3. **非确定性：**Agent 执行路径随采样参数、上下文变化而不同，同一输入可能产生不同执行路径；
 4. **中间状态评估：**需要评估 Agent 是否陷入了死循环、是否做出了不必要的工具调用；
 5. **安全维度：**Agent 能执行代码、调 API、读写文件，评测要覆盖安全边界。

- **评测维度扩展：**
- 除了最终答案质量，还要评：规划合理性、工具选择准确率、Loop 效率（轮次/Token 消耗）、安全合规得分。
- **评测方法：**
- 离线评测：用 Golden Set 跑批，评最终结果和关键中间步骤；
- Trace 回放：用生产 Trace 重新执行，看修复后结果是否一致；
- 线上灰度：小流量对比新旧 Agent，通过用户反馈和业务指标评判。

44 44. RAG、Agent、结构化输出的评测指标为什么不能混用一套？

44.1 参考答案要点

- **本质差异：**
- **RAG** 评的是”检索对没对 + 生成对没对”，关注忠实度、召回率；
- **Agent** 评的是”决策链对不对 + 执行到位没有”，关注任务成功率、工具调用准确性、执行效率；
- **结构化输出**评的是”格式合不合 Schema + 内容对不对”，关注 Schema 合规率、字段级准确率。
- **为什么不能混用：**
- 打分维度和权重完全不同：RAG 重视检索精度，Agent 重视规划合理性，结构化输出重视格式合规率；
- 失败模式不同：RAG 失败通常是检索不到文档，Agent 失败可能是死循环或工具调用错误，结构化输出失败是格式解析失败；
- 使用同一套 rubric 会导致打分不准确，无法定位具体问题。
- **建议：**三类场景分别建立评测集 + 评测 Prompt + 评测指标，再在 CI 流水线中串行执行。

五、LLM 基础与结构化输出（11 题）

45 45. Token 是什么？为什么中文、英文、代码消耗的 Token 不一样？

45.1 参考答案要点

- **Token 定义：**
- LLM 处理文本的最小单元，不等于”一个字”或”一个词”；
- 取决于分词器（Tokenizer），常见方式：BPE（Byte Pair Encoding）。
- **为什么消耗不同：**
- 英文：一个常见单词通常 1~2 个 Token（如”hello” = 1 token）；
- 中文：一个汉字通常 1~3 个 Token（如”你好”可能 = 2~6 tokens）；
- 代码：符号密集（括号、换行），每个符号通常 1 个 Token。
- **工程影响：**
- 中英文混合场景 Token 估算不准；
- 代码生成 Token 消耗比看起来大（缩进空格也是 Token）。

46 46. Temperature、Top-P、Top-K 分别控制什么？生产环境怎么设置更稳？

46.1 参考答案要点

参数	控制什么	生产建议
Temperature	概率分布的”陡峭度”，越低越确定	事实性任务 0~0.3，创意任务 0.7~1.0
Top-P	只考虑累积概率达到 P 的 token	0.9~0.95，过滤尾部低概率 token
Top-K	只考虑概率最高的 K 个 token	40~60，适合需要多样性但不过分发散

- **为什么 Temperature=0 输出仍可能不完全一致：**

- 浮点运算精度差异；
 - 不同硬件/框架的矩阵运算顺序可能不同；
 - 真正需要确定性应使用 seed 参数固定随机数生成器。
 - **生产建议：**
 - 事实性任务（RAG 回答、代码生成、数据提取）→ Temperature=0 配合固定 seed；
 - 开放式对话 → Temperature=0.7 + Top-P=0.9；
 - 不要三个参数同时调，先固定两个，调第三个看效果。
-

47 47. Streaming 为什么能改善用户体验？它能减少总耗时和 Token 成本吗？

47.1 参考答案要点

- **为什么能改善体验：**
 - 降低 TTFT (Time to First Token)：用户不用等完整回答，首字秒出；
 - 给予心理反馈：用户知道系统“在干活”，不是卡住了。
 - **不能减少什么：**
 - 总耗时不会减少（甚至可能稍长，因为多次网络传输开销）；
 - Token 成本不变（模型还是要生成完整回答）。
 - **技术选型：**
 - **SSE**：轻量、单向、HTTP 标准，适合普通 Chat 流式输出；
 - **WebSocket**：全双工、适合语音 Agent 和需要双向通信的场景；
 - **HTTP Chunked Transfer**：最底层方案，兼容性好但需要手动解析。
 - **工程注意：**
 - 断连重连：SSE 断连后需要重建上下文，可能造成 Token 浪费；
 - 取消流：用户中途打断，需要向 LLM API 发送取消信号停止计费。
-

48 48. JSON Mode 和 Structured Outputs 有什么区别？

48.1 参考答案要点

- **JSON Mode：**
 - 只保证输出是合法的 JSON 格式；
 - 不保证 JSON 的结构和字段符合约定 Schema；
 - 依赖 Prompt 中描述期望的字段，可靠性有限。
 - **Structured Outputs (JSON Schema)：**
 - 在 API 层面强约束输出必须符合给定的 JSON Schema；
 - 会修改模型采样逻辑，跳过不符合 Schema 的 token；
 - 可靠性远高于 JSON Mode，几乎 100% 格式合规。
 - **为什么只写“请返回 JSON”不可靠：**
 - LLM 本质是概率采样，无法保证每次都生成合法 JSON；
 - 嵌套结构或复杂字段时更容易出错；
 - 错误率随输出长度增加而上升。
 - **结构化输出失败怎么处理：**
 - 重试（带原输出和错误信息）；
 - 用正则/解析器尝试修复（如补全括号）；
 - 降级为非结构化回答（用户体验有损但系统可用）。
-

49 49. Function Calling 的完整链路是什么？和 MCP Agent Skill 是什么关系？

49.1 参考答案要点

- **Function Calling 完整链路：**
 1. 组装请求：System Prompt + 工具列表（JSON Schema 描述）+ 用户消息；
 2. LLM 推理：决定是否需要调用工具，选择工具，填写参数 JSON；
 3. 框架解析 LLM 输出中的 tool_call 字段；
 4. 执行工具函数，获取返回结果；
 5. 将工具结果追加到消息列表（role: tool）；
 6. 再次调用 LLM，让其基于工具结果继续推理或生成最终回答。
- **三者关系：**

	Function Calling	MCP	Agent Skill
层级	数据格式层	通信协议层	任务能力封装层
解决的问题	工具”长什么样”	工具”怎么连”	任务”怎么做一套”
粒度	单次工具调用	工具发现 + 调用	工具 + 流程 + 上下文 + 校验

- **一句话概括：**Function Calling定义方法签名，MCP 提供远程调用通道，Skill 把多个方法调用封装成可复用的任务模板。

50 50. 工具调用为什么必须做安全治理？安全边界在哪里？

50.1 参考答案要点

- **安全风险：**
 1. LLM 可能被诱导调用不该调的工具（Prompt 注入）；
 2. LLM 填写的参数可能不安全（文件路径穿越、SQL 注入、Shell 注入）；
 3. 工具调用结果可能包含敏感信息被带入后续上下文；
 4. 无人值守 Agent 可能反复调用昂贵接口（费用失控）；
 5. Agent 可能修改不应修改的代码或配置。
- **安全边界设计：**
 - **权限最小化：**每个 Agent 只能调用必需的、最小权限的工具集；
 - **参数校验：**工具函数内部做严格的参数校验（白名单、正则、范围检查）；
 - **二次确认：**高风险操作（删除、发布、数据库变更）必须人工确认；
 - **沙箱隔离：**代码执行在容器/沙箱中，限制文件系统和网络访问；
 - **审计日志：**记录每次工具调用的人参、结果、耗时和触发上下文；
 - **限流：**每个工具设置独立的调用频率和 Token 消耗上限。

51 51. 哪些大模型 API 错误可以重试？哪些不能重试？为什么必须做幂等？

51.1 参考答案要点

- **可重试错误：**
 - 429 Rate Limit：等待 Retry-After 后重试；
 - 5xx Server Error：瞬时故障，指数退避重试（最多 3 次）；
 - 网络超时/连接重置：重试。
- **不可重试错误：**
 - 401/403 鉴权失败：重试不会变好；
 - 400 参数错误：需修复请求后再试；

- 402 余额不足：需充值；
- 内容安全拦截（如 moderation flag）：换内容。
- 为什么必须做幂等：
- 网络波动可能导致请求已处理但响应丢失；
- 重试时服务端可能重复处理（重复扣费、重复执行副作用）；
- 实现方式：请求带上 idempotency key，服务端做去重。

52 52. 大模型为什么会产生幻觉？常见缓解方案有哪些？

52.1 参考答案要点

- 幻觉的根本原因：
 1. LLM 本质是概率模型，不是知识库——它在预测下一个 token，不是检索事实；
 2. 训练数据中的错误和不一致会被模型学习；
 3. 模型不知道”自己不知道”，强答不擅长的问题；
 4. 上下文信息不足或矛盾时，模型倾向自己”补全”。
- 缓解方案（从上到下优先级）：
 1. **RAG**：给模型注入权威的、可溯源的知识片段，最有效的缓释手段；
 2. **结构化约束**：用 JSON Schema / Structured Outputs 约束输出格式，减少自由发挥空间；
 3. **Prompt 引导**：明确告诉模型”不知道就说不知道”，禁止编造；
 4. **Truthfulness 校验**：用 LLM-as-Judge 对关键回答做事实核查；
 5. **人工审核**：高风险场景加入工 Review 环节。
- 不能完全消灭幻觉：RAG减少幻觉但不能根除，LLM 仍可能在推理步骤里生成错误判断。

53 53. Token 预算怎么估算？输入、输出、历史消息、RAG 证据如何取舍？

53.1 参考答案要点

- Token 预算公式：

1 上下文窗口剩余 = 模型最大上下文 - System Prompt Token - 历史对话 Token - RAG 证据 Token - 预留输出 Token

- 各部分取舍策略：
 - **System Prompt**：尽量精简（200~{}500 tokens），长规则放 SKILL.md 按需加载；
 - **历史对话**：滑动窗口保留最近 N 轮，更老的做摘要压缩；
 - **RAG 证据**：Top-3 到 Top-5 个 chunk，每个 256~{}512 tokens，总量控制 1500~{}2500 tokens；
 - **预留输出**：根据任务类型预留 500~{}2000 tokens 给模型回答。
- 预算超支处理：
 - 先削减 RAG 证据（减少 Top-K 或缩短 chunk）；
 - 再压缩历史对话（摘要 / 滑动窗口）；
 - 极端情况：切换到更大的上下文窗口模型。
- **实时监控**：每次调用前计算 Token 使用率，超过 40% 触发告警。

54 54. AI 应用调用日志里至少要记录哪些字段？为什么？

54.1 参考答案要点

- **最小必录字段：**
 - 基础：timestamp, trace_id, user_id, tenant_id, feature;
 - 请求：model, temperature, input_tokens, system_prompt_hash, rag_query;
 - 响应：output_tokens, total_tokens, latency_ms, ttft_ms, finish_reason;
 - 工具：tool_calls[] (name, params, result_summary, duration);
 - 错误：error_code, error_message, retry_count;
 - 成本：cost_usd, prompt_version。
- **为什么记录这些：**
 - 成本归因：按 user/tenant/feature/prompt_version 聚合；
 - 质量分析：finish_reason 区分正常完成/截断/内容过滤；
 - 故障排查：trace_id 串联全链路，error_code 定位故障点；
 - 优化基础：input/output tokens 对比优化效果。
- **日志设计原则：**
 - 结构化日志（JSON），不要用自由文本；
 - 敏感字段脱敏（用户消息内容 hash 后存储）。

55 55. 如果 LLM-as-Judge 和人工评测结果不一致，应该怎么处理？

55.1 参考答案要点

- **不一致的常见原因：**
 1. 评测 rubric 不够具体，“好”的标准模糊；
 2. LLM-as-Judge 有系统性偏差（位置偏差、冗长偏好等）；
 3. 人工评测者有自己的主观偏好；
 4. 评测样本数量不足，结论不具有统计意义。
- **处理流程：**
 1. **抽样对比：**从不一致的样本中随机抽 20~30 条，由多位人工评测者独立打分；
 2. **找出分歧模式：**LLM-as-Judge 是否系统性地高估/低估某一类回答？
 3. **调整 rubric：**细化评测维度（如把“回答质量”拆成忠实度、相关性、准确性三个子维度）；
 4. **校准评测 Prompt：**在 LLM-as-Judge 的 prompt 中针对发现的偏差加反例约束；
 5. **计算 IAA：**计算 LLM-as-Judge 与人工评测的 Inter-Annotator Agreement，目标 > 0.7；
 6. **持续迭代：**每次发现新的偏差类型后，更新评测 Prompt 和 rubric。
- **兜底方案：**高权重决策（如安全、合规）最终以人工评测为准；LLM-as-Judge 用于快速过滤和趋势监控。

56 附录：面试答题框架建议

在面试中回答 AI Agent 相关问题，建议采用以下框架：

1. **先给定义：**一句话说清楚这个技术是什么；
2. **再讲原理：**为什么需要它，解决什么问题；
3. **对比分析：**和相近技术有什么区别（如 RAG vs Memory | MCP vs Function Calling | Agent vs Workflow）；
4. **工程实践：**怎么落地，注意什么坑；
5. **Tradeoff：**有什么取舍，什么场景适用，什么场景不适用。

一条重要的面试金句：Agent = Model + Harness。模型负责推理和生成，Harness 负责环境、工具、反馈、沙箱、权限、观测和恢复。决定 Agent 表现上限的，往往不是模型有多强，而是 Harness 有多稳。